Traceability

References

- Best Practices for Maintaining a Requirement Traceability Matrix in Agile - Ketryx Compliance Framework
- The importance of traceability - Work Life by Atlassian
- Requirements Traceability Matrix in Six Sigma: A Complete Guide for Project Success - SixSigma.us

Definition:

1.

> Requirement traceability, ... , refers to the ability to trace each requirement back to its origin and to its implementation and testing. These pathways ensure that all software components are linked to corresponding business needs and help teams monitor progress, validate completeness, and identify any gaps in requirements coverage.
>
> Best Practices for Maintaining a Requirement Traceability Matrix in Agile - Ketryx Compliance Framework

2.

> “Traceability” is the ability to describe and follow the life of a requirement in both a forwards and backwards direction – from requirements-to-code, and vice versa. When executed effectively, traceability allows you to track work items across the development lifecycle to where requirements are implemented in the code, playing a crucial role in your organization’s control framework.
>
> The importance of traceability - Work Life by Atlassian

Key Topics

Requirements Traceability Life Cycle

- ✅ Requirements traceability matrix in an Agile process – Webomates

Contents of a Traceability Matrix

- ✅ Requirements traceability matrix in an Agile process – Webomates
- The Ultimate Guide to Requirements Traceability Matrix (RTM)

Three types of Requirements Traceability Matrices

- ✅ 6 Requirements Traceability Matrix in Six Sigma: A Complete Guide for Project Success - SixSigma.us

Implementing Traceability

- ✅ The importance of traceability - Work Life by Atlassian
  Excellent – shift-left: Give the developer feedback as early as possible if his or her code is not traceable. High-performing engineering teams are doing this as part of the CI process, but excellent engineering teams add indication at an even earlier stage, when the code is pushed to the repository.
- What if our test suite directory structure mirrored the TI > Epic > Story structure, and when a test is created it is placed inside a directory tree like this, for example `TI-123`, Epic `DUCKS-4567`, Story `DUCKS-8901`

  ```
  specs/plugins/pal/TI-123/DUCKS-4567/DUCKS-8901/new_test.spec.ts
  ```

  And when this tests gets updated/modified in a future story `DUCKS-9000` (assuming the same Epic) we update the test's path like this:

  ```
  specs/plugins/pal/TI-123/DUCKS-4567/DUCKS-9000/new_test.spec.ts
  ```

- In addition, (or alternatively) we can add comments with this same TI > Epic > Story structure to our tests, in three distinct scenarios
  - New `.spec.ts` file, the entire spec is tagged with a comment

    ```
    specs/plugins/pal/TI-123/DUCKS-4567/DUCKS-8901/new_test.spec.ts
    =========

      1 /*
      2  *                                     Copyright 2024 WebPros International, LLC
    ```

```
 3  *                                                    All rights reserved.
 4  * copyright@cpanel.net                                  http://cpanel.net
 5  * This code is subject to the cPanel license. Unauthorized copying is prohibited.
 6  */
 7
 8 /*
 9  * TI-123/DUCKS-4567/DUCKS-8901
10  */
11
12 import { test, expect } from "@fixtures/plugins/pal/pal";
```

- New/updated `test()` case(s) (corresponding to story `DUCKS-9000`) inside of an existing `spec.ts` get(s) tagged with a comment

```
specs/plugins/pal/TI-123/DUCKS-4567/DUCKS-8901/new_test.spec.ts
=========

 1 /*
 2  *                                      Copyright 2024 WebPros International, LLC
 3  *                                                    All rights reserved.
 4  * copyright@cpanel.net                                  http://cpanel.net
 5  * This code is subject to the cPanel license. Unauthorized copying is prohibited.
 6  */
 7
 8 /*
 9  * TI-123/DUCKS-4567/DUCKS-8901
10  */
11
12 import { test, expect } from "@fixtures/plugins/pal/pal";

.
.
.

49     /*
50      * TI-123/DUCKS-4567/DUCKS-9000
51      */
52     test("New test case", async ({
```

- New/updated `expect()` assertion(s) (corresponding to story `DUCKS-9001`) inside of an existing `test()` case get(s) tagged with a comment

```
specs/plugins/pal/TI-123/DUCKS-4567/DUCKS-8901/new_test.spec.ts
=========

49     /*
50      * TI-123/DUCKS-4567/DUCKS-9000
51      */
52     test("New test case", async ({

.
.
.

            /*
108          * TI-123/DUCKS-4567/DUCKS-9001
109          */
110         await expect(
111             blah,
112             "Blah is visible"
113         ).toBeVisible();
```

- This ⬆️ scheme can be referred to as **Scoped Traceability Coments**, and we can easily `grep` for the set of `test()`s and `expect()`s related to a given Epic or Story in order to verify coverage.
- This ⬆️ can also be used in tandem with the Jira/Atlassian CLI and Jira JQL syntax to answer questions like “What is our test coverage for the completed stories under Epic DUCKS-4567?“ like so:

```
acli jira workitem search --jql "parent=DUCKS-4567 status=Done" /
perl -E "...process this ^ output to return only story IDs"     /
| while read storyID ; grep storyID specs/plugins/pal/
```